

CS5236 - Advanced Automata Theory
Audit
Summary

Sim Ray En Ryan

December 14, 2025

Contents

1	Chomsky Hierarchy and Grammars	4
1.1	Formal Languages and Operations	4
1.2	Mechanisms to Describe a Language	4
1.3	Regular Expressions and Languages	5
1.4	Grammars	5
1.5	The Chomsky Hierarchy (Definition 1.11)	5
1.6	Equivalence of Regular Grammars and Regular Expressions	6
1.7	Pumping Lemmas	6
2	Finite Automata	7
2.1	Introduction to Finite Automata	7
2.2	Deterministic Finite Automaton (DFA) (Description 2.3)	7
2.3	Characterising Regular Sets	7
2.4	Block Pumping Lemma	8
2.5	Myhill-Nerode Theorem and Minimal DFA	8
2.6	Jaffe's Matching Pumping Lemma	8
2.7	Nondeterministic Finite Automaton (NFA)	8
2.8	Determinisation of NFAs	9
2.9	Summary of Regular Language Equivalences	9
3	Combining Languages	9
3.1	Basic Closure Properties	9
3.2	Closure of Regular Languages using DFA Product Automata	10
3.3	Closure of Context-Free Languages (CFLs)	10
3.4	Closure of Context-Sensitive Languages (CSLs)	10
3.5	Linear Languages	11
4	Games on Finite Graphs	11
4.1	Finite Graph Games	11
4.2	Algorithm for Finite Games	11
4.3	Alternating Finite Automaton (AFA)	12
5	Games Played for an Infinite Time	12
5.1	Survival Games	12
5.2	Update Games	13
5.3	Büchi Games	13
5.4	Parity Games	13
6	Automata on Infinite Sequences (ω-Words)	13
6.1	Büchi Automata	13
6.2	Büchi's Characterisation	14
6.3	Muller Automata	14

6.4	Rabin and Streett Automata	14
7	Automatic Functions and Relations	15
7.1	Definition of Automaticity	15
7.2	Closure Properties	15
8	Groups, Monoids and Automata Theory	15
8.1	Algebraic Definitions	15
8.2	Syntactic Monoids and Regularity	16
8.3	Automatic Groups and Semigroups	16
9	Automatic Structures in General	16
9.1	Definition	16
9.2	Properties and Limitations	17
10	Transducers and Rational Relations	17
10.1	Rational Relations	17
10.2	Transducers (Rational Functions)	17

1 Chomsky Hierarchy and Grammars

1.1 Formal Languages and Operations

- A **language** L is usually a set of strings w over an alphabet Σ . The alphabet Σ is usually finite.
- **Concatenation:** vw or $v \cdot w$ denotes the concatenation of strings v and w .
- **Set Definitions (Definition 1.1):**
 - **Finite List:** E.g., $\{001, 0011, 00111\}$.
 - **Kleene Star (L^*):** The set of all strings obtained by concatenating finitely many strings from L . $L^* = \{u_1 \cdot u_2 \cdot \dots \cdot u_n : n \in \mathbb{N} \wedge u_1, u_2, \dots, u_n \in L\}$.
 - **Union ($L \cup H$):** The set of all strings in L or in H .
 - **Intersection ($L \cap H$):** The set of all strings in L and in H .
 - **Concatenation (LH):** The set $\{v \cdot w : v \in L \wedge w \in H\}$.
 - **Set Difference ($L - H$):** The set $\{u : u \in L \wedge u \notin H\}$.
- **Special Symbols (Remarks 1.2):**
 - $\emptyset$: The empty set ($\{\}$).
 - ϵ : The empty string.
 - $\{\epsilon\}$: The set containing the empty string.
 - Σ^* is the set of all strings over the alphabet Σ .
 - $\emptyset^* = \{\epsilon\}^* = \{\epsilon\}$.
 - L^+ : The set of all non-empty concatenations over members of L . Note: $L^+ = L \cdot L^*$.
- **Formal Languages:** Languages L for which there is a mechanism to check membership or to generate all members.

1.2 Mechanisms to Describe a Language

- **Automaton/Machine:** Checks whether a given word w belongs to L .
- **Grammar:** Generates all words w belonging to L using step-wise rules.
- **Transducer:** A function f that translates words to words such that L is the image of another language H under f .
- **Expression:** Describes the language in a short-hand (e.g., Regular Expressions).

1.3 Regular Expressions and Languages

- **Regular Expression (Definition 1.3):** Denotes either a finite set, the empty set ($\emptyset$), or is formed from other regular expressions by the operations of Kleene star ($*$), concatenation ($\cdot$), and union ($\cup$). Intersection and set difference are sometimes omitted, as they can be defined using other operations.
- **Regular Languages:** Languages that can be defined using regular expressions.

1.4 Grammars

- **Grammar (Definition 1.6):** A structure (N, Σ, P, S) where:
 - N : Set of **non-terminal** symbols.
 - Σ : Set of **terminal** symbols ($\Sigma \cap N = \emptyset$).
 - P : A set of **rules** (productions) of the form $l \rightarrow r$, where l is a string containing at least one symbol from N .
 - $S \in N$: The **starting symbol**.
- **Derivation:**
 - $v \Rightarrow w$: w can be derived from v in **one step**.
 - $v \Rightarrow^* w$: w can be derived from v in an **arbitrary number of steps**.
- **Language Generated (L):** The set $L = \{w \in \Sigma^* : S \Rightarrow^* w\}$.
- **Example (Binary Numbers):** $S \rightarrow 0 \mid 1T$, $T \rightarrow T0 \mid T1 \mid \epsilon$. S and T are non-terminals; 0, 1 are terminals.

1.5 The Chomsky Hierarchy (Definition 1.11)

- The hierarchy classifies formal grammars and the languages they generate.
- **(CH3) Regular Grammar / Right-Linear:** Every rule is of the form $A \rightarrow wB$ or $A \rightarrow w$, where $A, B \in N$ and $w \in \Sigma^*$.
 - **Regular Language:** Generated by a regular grammar.
- **(CH2) Context-Free Grammar:** Every rule is of the form $A \rightarrow w$, where $A \in N$ and $w \in (N \cup \Sigma)^*$.
 - **Context-Free Language:** Generated by a context-free grammar.
- **(CH1) Context-Sensitive Grammar:** Every rule is of the form $uAw \rightarrow uvw$, where $A \in N$, $u, v, w \in (N \cup \Sigma)^*$, and $v \neq \epsilon$. The rule $S \rightarrow \epsilon$ is allowed only if S does not appear on the right side of any rule.

- **Context-Sensitive Language:** Generated by a context-sensitive grammar.
- **Alternative Characterisation (Theorem 1.12):** A language $L \neq \{\epsilon\}$ is context-sensitive iff it can be generated by a grammar where every rule $l \rightarrow r$ satisfies $|l| \leq |r|$.
- **(CH0) General Grammar:** No restrictions beyond the basic definition.
 - **Recursively Enumerable Language:** Generated by some general grammar.

1.6 Equivalence of Regular Grammars and Regular Expressions

- **Theorem 1.14:** A language L is generated by a regular expression if and only if it is generated by a regular grammar.

1.7 Pumping Lemmas

- **Purpose:** Methods to show that certain languages are **not** regular or **not** context-free. They are sufficient but not necessary criteria.
- **Pumping Lemma for Regular Languages (Theorem 1.19a):**
 - If L is an infinite regular language, there is a constant k such that for every $u \in L$ with $|u| \geq k$, there is a representation $u = xyz$ such that:
 - * $|xy| \leq k$
 - * $y \neq \epsilon$
 - * $xy^*z \subseteq L$ (i.e., $xy^l z \in L$ for all $l \in \mathbb{N}$)
- **Pumping Lemma for Context-Free Languages (Theorem 1.19b):**
 - If L is an infinite context-free language, there is a constant k such that for every $u \in L$ with $|u| \geq k$, there is a representation $u = vwx y z$ such that:
 - * $|wxy| \leq k$
 - * $w \neq \epsilon \vee y \neq \epsilon$
 - * $vw^l xy^l z \in L$ for all $l \in \mathbb{N}$
- **Corollary 1.20 (Weaker Regular Pumping):** If L is an infinite regular language, there is a constant k such that for each $w \in L$ with $|w| > k$, one can represent $w = xyz$ with $y \neq \epsilon$ and $xy^*z \subseteq L$. (Note: Unlike Theorem 1.19a, this version does not require $|xy| \leq k$).

2 Finite Automata

2.1 Introduction to Finite Automata

- An **automaton** is a mechanism that checks whether a word is in a given language.
- It has a finite number of **states** (memory values) that store information.
- **Accepting States (Final States)**: States that indicate the word is in the language.
- **Rejecting States**: States that indicate the word is not in the language.
- **Initial State (Starting State)**: The starting value of the memory.
- **Graphical Representation**: States are nodes, accepting states have a double border, the initial state is marked by an incoming arrow, and transitions are labelled arrows.

2.2 Deterministic Finite Automaton (DFA) (Description 2.3)

- **Formal Definition**: A DFA is a tuple $M = (Q, \Sigma, \delta, s, F)$, where:
 - Q : The finite set of **states**.
 - Σ : The **alphabet** (a finite set of symbols).
 - δ : The **transition function** $\delta : Q \times \Sigma \rightarrow Q$. It maps a state and an input symbol to a **unique** next state.
 - $s \in Q$: The **starting state**.
 - $F \subseteq Q$: The set of **accepting states**.
- **Extended Transition Function**: $\delta(q, w)$ is extended inductively to operate on strings $w \in \Sigma^*$.
 - $\delta(q, \epsilon) = q$
 - $\delta(q, wa) = \delta(\delta(q, w), a)$ for $a \in \Sigma$ and $w \in \Sigma^*$
- **Acceptance**: An automaton accepts a word $w \in \Sigma^*$ iff $\delta(s, w) \in F$.

2.3 Characterising Regular Sets

- **Theorem 2.8 (DFA $\implies$ Regular Language)**: If a language L is recognised by a deterministic finite automaton, then L is regular (i.e., generated by a regular grammar).
- **Theorem 1.14 + Theorem 2.42 (Regular Language $\implies$ NFA $\implies$ DFA)**: The converse is also true: every language generated by a regular grammar is recognised by a DFA.

2.4 Block Pumping Lemma

- **Theorem 2.9 (Block Pumping Lemma):** If L is a regular set, there is a constant k such that for all strings $u_0u_1 \dots u_k \in L$ where $u_1, \dots, u_{k-1}$ are non-empty, there exist indices i, j with $0 < i < j \leq k$ and:

$$(u_0u_1 \dots u_{i-1}) \cdot (u_iu_{i+1} \dots u_{j-1})^* \cdot (u_ju_{j+1} \dots u_k) \subseteq L$$

- **Mechanism (Proof):** The constant k can be set to $|Q|+1$ where $|Q|$ is the number of states in the recognizing DFA. By the Pigeonhole Principle, in the sequence of states entered after reading $u_0, u_0u_1, \dots, u_0u_1 \dots u_k$, two states must be the same.
- **Theorem 2.11 (Ehrenfeucht, Parikh, Rozenberg):** A language L is regular if and only if both L and its complement satisfy the block pumping lemma.

2.5 Myhill-Nerode Theorem and Minimal DFA

- **Derivative of a Language (L_x):** $L_x = \{y \in \Sigma^* : xy \in L\}$.
- **Theorem 2.19 (Myhill and Nerode's Minimal DFA):** The language L is regular if and only if the number of different derivatives L_x is finite.
- **Minimal DFA:** For a language with exactly n distinct derivatives, one can construct a complete DFA having n states, and no complete DFA with fewer than n states can recognize L .

2.6 Jaffe's Matching Pumping Lemma

- **Lemma 2.23 (Jaffe's Matching Pumping Lemma):** A language $L \subseteq \Sigma^*$ is regular if and only if there is a constant k such that for all $x \in \Sigma^*$ and $y \in \Sigma^k$, there are u, v, w with $y = uvw$ and $v \neq \epsilon$ such that, for all $h \in \mathbb{N}$, the derivatives are equal: $L_{xuv^hw} = L_{xy}$.
- **Significance:** This provides a necessary and sufficient condition for regularity (unlike the standard Pumping Lemma). The equality of derivatives implies that the "pumping" property holds not only for membership in L but in a "matching" way for all future extensions (z).

2.7 Nondeterministic Finite Automaton (NFA)

- **Definition 2.32:** An NFA is a tuple $(Q, \Sigma, \delta, s, F)$, differing from a DFA in that the transition function δ is multi-valued, i.e., for each $q \in Q$ and $a \in \Sigma$, $\delta(q, a)$ is a set of states.
- **Run:** A sequence of states $q_0q_1 \dots q_n$ on input $a_1 \dots a_n$ where $q_0 = s$ and $q_{m+1} \in \delta(q_m, a_{m+1})$.

- **Acceptance:** An NFA accepts a word w iff there is at least one accepting run (a run whose last state is in F).
- **Size Difference (Example 2.33):** An NFA can be exponentially smaller than the equivalent DFA (e.g., $n + 2$ states vs. $2^n + 1$ states for a certain language).

2.8 Determinisation of NFAs

- **Theorem 2.34 (Determinisation of NFAs):** For every NFA with n states, there is an equivalent DFA with up to 2^n states.
- **Construction (Subset Construction):** The states of the DFA are the subsets Q' of the NFA states Q .
 - Starting state: $\{s\}$.
 - Transition function δ' : $\delta'(Q', a) = \{q'' \in Q : \exists q' \in Q' [q'' \in \delta(q', a)]\}$ (The set of all states reachable from any state in Q' upon reading a).
 - Accepting states F' : $\{Q' \subseteq Q : Q' \cap F \neq \emptyset\}$.

2.9 Summary of Regular Language Equivalences

- **Corollary 2.46:** A language L is regular if and only if it satisfies any of the following equivalent conditions:
 - L is generated by a **regular expression**.
 - L is generated by a **regular grammar**.
 - L is recognised by a **deterministic finite automaton (DFA)**.
 - L is recognised by a **nondeterministic finite automaton (NFA)**.
 - L and its complement satisfy both the **block pumping lemma**.
 - L satisfies **Jaffe's pumping lemma**.
 - L has only **finitely many derivatives** (Myhill-Nerode Theorem).

3 Combining Languages

3.1 Basic Closure Properties

- **Theorem 3.1 (Basic Closure Properties):** If L, H are languages at level CH_k of the Chomsky hierarchy, then $L \cup H$, LH , and L^* are also at level CH_k . This holds for all four levels (CH0 to CH3).

3.2 Closure of Regular Languages using DFA Product Automata

- ****Description 3.2 (Product Automata):**** Given DFAs for L_1 and L_2 , a product automaton $(Q_1 \times Q_2, \Sigma, \delta_1 \times \delta_2, (s_1, s_2), F)$ can recognize the union, intersection, or difference by selecting the final states F :
 - ****Intersection $(L_1 \cap L_2)$:**** $F = F_1 \times F_2$. (Both must accept)
 - ****Union $(L_1 \cup L_2)$:**** $F = F_1 \times Q_2 \cup Q_1 \times F_2$. (At least one must accept)
 - ****Difference $(L_1 - L_2)$:**** $F = F_1 \times (Q_2 - F_2)$. (First must accept, second must reject)
 - ****Symmetric Difference:**** $F = F_1 \times (Q_2 - F_2) \cup (Q_1 - F_1) \times F_2$. (Exactly one must accept)

3.3 Closure of Context-Free Languages (CFLs)

- ****Closure under Union, Concatenation, Kleene Star:**** CFLs are closed under these operations (Exercise 3.7). The new grammar is constructed by introducing a new start symbol and non-terminal rules based on the operation.
- ****Non-closure under Intersection/Complement:****
 - ****Intersection:**** CFLs are **not** closed under intersection. E.g., $L = \{0^n 1^n 2^n : n \in N\}$ is not context-free, but it is the intersection of two CFLs: $\{0^n 1^n 2^* : n \in N\}$ and $\{0^* 1^n 2^n : n \in N\}$ (Example 3.8).
 - ****Complement:**** CFLs are **not** closed under complement (because $\overline{L_1 \cap L_2} = \overline{L_1} \cup \overline{L_2}$).
- ****Closure under Intersection with Regular Languages (Theorem 3.9):****
 - If L is a context-free language and H is a regular language, then $L \cap H$ is also a context-free language.
 - ****Proof Sketch:**** Construct a new grammar where non-terminals encode the state information of the DFA recognizing H alongside the original non-terminals of the context-free grammar for L . The new non-terminals are of the form (p, A, q) , representing the derivation of a string w from A in L while simultaneously moving the DFA from state p to q on w .

3.4 Closure of Context-Sensitive Languages (CSLs)

- ****Concatenation (Theorem 3.12):**** The concatenation of two context-sensitive languages is context-sensitive. (Use a new start symbol $S \rightarrow S_1 S_2$ with disjoint non-terminals and combine the rules, maintaining the length-non-decreasing property).
- ****Kleene Star (Theorem 3.13):**** If L is context-sensitive, so is L^* .

- **Intersection (Theorem 3.15):** The intersection of two context-sensitive languages is context-sensitive. (Uses a complex proof sketch involving non-terminals as pairs of symbols, $((\begin{smallmatrix} A \\ B \end{smallmatrix}))$, to simulate two grammars in parallel on matching input components).
- **Complement:** The complement of a context-sensitive language is context-sensitive (Theorem 7.13, Immerman-Szelepcsényi).

3.5 Linear Languages

- **Linear Grammar:** Every rule is either of the form $A \rightarrow u$ or $A \rightarrow vBw$, where A, B are non-terminals and u, v, w are terminal words.
- **Linear Language:** A language generated by a linear grammar. E.g., the language of palindromes.
- **Closure under Intersection with Regular Languages (Exercise 3.30):** The intersection of a linear language and a regular language is linear.

4 Games on Finite Graphs

4.1 Finite Graph Games

- **Setup:** A finite graph with nodes Q and a set of target nodes $T \subseteq Q$. Two players (Anke and Boris) move a marker alternately. The player who moves the marker first into a target node wins.
- **Winning Strategy:** An algorithm that tells a player how to move in every position (based on prior moves) to ensure a win.
- **Winning Position:** A node v from which the current player has a winning strategy.
- **Draw Games:** Games where the starting position is not a winning position for either player (can result in infinite play).

4.2 Algorithm for Finite Games

- **Theorem 4.6:** There is a polynomial-time algorithm to determine which player has a winning strategy from any given node.
- **Algorithm Steps (The maximum number of moves to a win):**
 1. Start with $T_0 = T$ (Boris winning in 0 moves) and $S_0 = \emptyset$.
 2. S_{n+1} : Nodes from which Anke can move in one step to a node in T_n . (Anke wins in $\leq n + 1$ moves).

3. T_{n+1} : Nodes (not in $T_n \cup S_{n+1}$) from which all moves lead to a node in S_{n+1} . (Boris wins in $\leq n + 1$ moves).
 4. Repeat until no new nodes are added.
- **Result:** The final S_n are winning positions for Anke, and $T_n - T$ are winning positions for Boris. The remaining nodes $R = Q - (T_n \cup S_n)$ are **draw positions** (players can force an infinite game within R).

4.3 Alternating Finite Automaton (AFA)

- **Definition 4.15 (AFA):** A finite automaton where transitions on (q, a) can be:
 - **Deterministic:** $\rightarrow q'$ (unique successor).
 - **Disjunctive ($\vee$):** $q' \vee q'' \vee \dots$ (Anke picks a successor).
 - **Conjunctive ($\wedge$):** $q' \wedge q'' \wedge \dots$ (Boris picks a successor).
- **Acceptance:** An AFA accepts a word w iff Anke has a winning strategy for w , regardless of Boris's moves.
- **AFA to NFA (Theorem 4.17):** An AFA with n states is equivalent to an NFA with up to 2^n states.
- **Blow-up (AFA to DFA):** The transition from an AFA to an equivalent DFA can involve a double-exponential blow-up in the number of states (Example 4.18).
- **Intersection with AFA (Theorem 4.20):** Given n DFAs (or NFAs), their intersection can be recognized by a smaller AFA than a product automaton. The AFA's initial transition uses a conjunction to distribute the task to separate DFA simulations.

5 Games Played for an Infinite Time

5.1 Survival Games

- **Definition 5.1 (Survival Game):** An infinite game where a set of nodes are considered "bad" (dead ends). Player Anke wins if the play runs forever without entering a bad node (i.e., survives forever). Boris wins if the game eventually ends in a bad node.
- **Winning Strategy:** For a survival game, either Anke or Boris always has a **memoryless winning strategy** (the move depends only on the current state).
- **Decision (Theorem 5.4):** There is a polynomial-time algorithm to determine which player wins a survival game by computing a partial function $f(v, \text{player})$ rep-

representing the minimum number of moves needed by the opposing player to force a bad state. $f(v, \text{player}) = \infty$ indicates a win for the current player.

5.2 Update Games

- **Definition 5.6 (Update Game):** An infinite game where Anke wins iff she visits every node in a special set W infinitely often.
- **Winning Strategy:** An update game may require a **memory-based winning strategy** for Anke (Example 5.7).
- **Decision (Theorem 5.8):** There is an algorithm (polynomial time, assuming sub-routines are polynomial) to determine the winner based on two conditions which check if a player can force the game into a region v and then force repeated cycles through all $w \in W$ and back to v .

5.3 Büchi Games

- **Description 5.11 (Büchi Game):** An infinite game similar to an update game, but Anke wins iff she visits **one or more** nodes in W infinitely often.
- **Decision (Theorem 5.12):** There is a polynomial-time algorithm that determines the winner. The winning strategies for Büchi games are always memoryless.

5.4 Parity Games

- **Description 5.13 (Parity Game):** An infinite game where each node $v \in V$ is assigned a value $\text{val}(v)$. Anke wins a play $v_0, v_1, \dots$ iff the limit superior (largest value visited infinitely often) of the sequence $\text{val}(v_0), \text{val}(v_1), \dots$ is an **even** number.
- **Winning Strategy:** In a parity game, one of the players always has a **memoryless winning strategy**.
- **Decision (Open Problem 12.24/Theorem 12.20):** Deciding the winner of a parity game is an important open problem. Deterministic algorithms with sub-exponential running times are known, such as $O(n^{c \log m})$ where n is the number of nodes and m is the maximum value.

6 Automata on Infinite Sequences (ω -Words)

6.1 Büchi Automata

- **Description 6.1 (Büchi Automata):** A nondeterministic automaton $(Q, \Sigma, \delta, s, F)$ for ω -words $(b_0 b_1 b_2 \dots \in \Sigma^\omega)$.

- ****Acceptance:**** A run $q_0q_1q_2 \dots$ on an ω -word is accepting iff the set of infinitely often visited states $U = \{p \in Q : \exists^\infty k [q_k = p]\}$ intersects the set of final states F ($U \cap F \neq \emptyset$).
- ****Deterministic vs. Nondeterministic:**** Unlike finite automata, nondeterministic Büchi automata are strictly more powerful than deterministic ones (Example 6.5).

6.2 Büchi's Characterisation

- ****Theorem 6.14 (Büchi's Characterisation):**** An ω -language L is recognized by a nondeterministic Büchi automaton if and only if it can be expressed as a finite union of concatenations of a regular language and the ω -Kleene star of a regular language:

$$L = \bigcup_{m=1}^n A_m B_m^\omega$$

where A_m and B_m are regular languages (and B_m contains only non-empty words).

6.3 Muller Automata

- ****Description 6.15 (Muller Automaton):**** A generalisation of Büchi automata, defined by $(Q, \Sigma, \delta, s, G)$, where G is a set of subsets of Q .
- ****Acceptance:**** A run is accepting iff the set of infinitely often visited states U belongs to G ($U \in G$).
- ****Equivalence (Theorem 6.19 - McNaughton's Characterisation):**** The following are equivalent for an ω -language L :
 - L is recognized by a ****nondeterministic Büchi automaton****.
 - L is recognized by a ****deterministic Muller automaton****.
 - L is recognized by a ****nondeterministic Muller automaton****.
- ****Blow-up:**** The translation from a nondeterministic Büchi automaton to a deterministic Muller automaton involves an exponential blow-up in the number of states.

6.4 Rabin and Streett Automata

- ****Description 6.24 (Rabin/Streett):**** Automata with an acceptance condition Ω consisting of pairs of state sets, (E, F) .
- ****Rabin Automaton Acceptance:**** A run is accepting if U satisfies $U \cap E \neq \emptyset$ and $U \cap F = \emptyset$ for at least one pair $(E, F) \in \Omega$.
- ****Streett Automaton Acceptance:**** A run is accepting if U satisfies $U \cap E \neq \emptyset$ or $U \cap F = \emptyset$ for all pairs $(E, F) \in \Omega$.

- **Complement:** The deterministic Streett automaton for $\Omega' = \{(F, E) : (E, F) \in \Omega\}$ recognizes the complement of the language recognized by the deterministic Rabin automaton for Ω .

7 Automatic Functions and Relations

7.1 Definition of Automaticity

- **Definition 7.1 (Automatic Relations/Functions):**
 - A relation $R \subseteq X \times Y$ is **automatic** iff there is a DFA that reads both inputs simultaneously (one symbol per cycle) at the same speed, padding the shorter input with a special symbol $\#$. The input string for the DFA is the convolution of x and y , $\text{conv}(x, y)$. The relation holds iff the DFA is in an accepting state when both inputs are completely read.
 - A function $f : X \rightarrow Y$ is **automatic** iff the relation $\{(x, y) : x \in \text{dom}(f) \wedge y = f(x)\}$ is automatic.
- **Notation 7.3 (Convolution):** $\text{conv}(x, y)$ is the string of column vectors, e.g., $\text{conv}(00, 111) = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} \# \\ 1 \end{pmatrix}$.

7.2 Closure Properties

- **Theorem 7.7 (First-Order Definable Relations):** If a relation (or function) is first-order definable using automatic functions and relations, then it is also automatic.
- **Regular Image:** For an automatic function f and a regular set $R \subseteq \text{dom}(f)$, the image $f(R) = \{f(x) : x \in R\}$ is a regular set: $y \in f(R) \Leftrightarrow \exists x \in R[f(x) = y]$.

8 Groups, Monoids and Automata Theory

8.1 Algebraic Definitions

- **Definition 8.1:**
 - **Semigroup:** A set G with an associative operation $\circ$.
 - **Monoid:** A semigroup $(G, \circ)$ with a neutral element e .
 - **Group:** A monoid with an inverse element for every $x \in G$.
 - **Finitely Generated:** There is a finite set of generators F such that every element $x \in G$ can be expressed as $x = y_1 \circ \dots \circ y_n$ with $y_i \in F$.

8.2 Syntactic Monoids and Regularity

- **Example 8.5 (DFA Monoid):** The set of functions $Q \rightarrow Q$ realized by words $w \in \Sigma^*$ in a complete DFA forms a finite monoid of transformation functions, $\text{Syn}(M)$.
- **Congruence Relation:** A DFA defines a congruence relation $v \sim w$ iff $f_v = f_w$ (the transformations realized by v and w are identical).
- **Syntactic Monoid of a Language L (G_L):** The finite monoid derived from the minimal DFA recognizing L .
- **Theorem 8.6:** A language is regular if and only if it is the finite union of equivalence classes of some congruence relation with finitely many congruence classes.

8.3 Automatic Groups and Semigroups

- **Description 8.15 (Automatic in the sense of Epstein, Cannon, ...):**
 - A finitely generated semigroup G' is **automatic** if it is isomorphic to a semigroup $G \subseteq F^*$ (where F is the set of generators) such that:
 1. The set of representatives G is a **regular subset** of F^* .
 2. Each element has exactly one representative in G .
 3. For each fixed generator $a \in F$, the left-multiplication map $x \mapsto \text{el}_G(x \cdot a)$ is an **automatic function**.
 - **Biautomatic:** If both left- and right-multiplication by generators are automatic.
- **Description 8.19 (Fully Automatic in the sense of Hodgson, Khoussainov and Nerode):**
 - A structure $(G, \circ)$ is **fully automatic** if G is a **regular subset** of Σ^* and the full semigroup operation $\circ : G \times G \rightarrow G$ is an **automatic function**.

9 Automatic Structures in General

9.1 Definition

- **Definition 9.1 (Automatic Structure):** A structure $(A, R_1, \dots, R_h, f_1, \dots, f_k)$ is called **automatic** iff:
 - A (the domain) is a **regular subset** of some Σ^* .
 - All relations R_i and functions f_j are **automatic**.

9.2 Properties and Limitations

- **Non-Automatic Structures:**
 - There is **no infinite automatic field** (Theorem 9.7).
 - The multiplicative monoid of the natural numbers is not automatic.
 - The set of all palindromes Pal is not regular, hence $(\{0, 1\}^*, \text{Pal}, \dots)$ is not automatic in the given representation (Exercise 9.19).
- **Automatic Ordinals (Theorem 9.10):** The structure of ordinals strictly below α with their ordering $<$ is automatic if and only if $\alpha < \omega^\omega$.

10 Transducers and Rational Relations

10.1 Rational Relations

- **Definition 10.1 (Rational Relation):** A relation $R \subseteq (\Sigma^*)^n$ is rational if it is recognized by a nondeterministic finite state machine that reads n inputs in parallel, but **at different speeds** (asynchronously). Transitions are labeled by an n -tuple of words $(w_1, w_2, \dots, w_n)$, where $|w_k|$ symbols are consumed from the k -th input.
- **Examples of Rational Relations:**
 - String Concatenation: $(x, y, z) \in R \Leftrightarrow z = xy$.
 - Subsequence Relation: x is a subsequence of y .
- **Automatic vs. Rational:** Every automatic relation is also a rational relation, but not vice versa (rational relations are also called asynchronously automatic).

10.2 Transducers (Rational Functions)

- **Definition 10.6 (Transducers):** A mechanism to compute a rational function $f : \Sigma^* \rightarrow \Sigma^*$.
- **Mealy Machine:** An NFA where each transition is attributed with a pair (v, w) (input part v , output part w).
- **Moore Machine (Description 10.7):** An NFA where each state q owns a word w_q . An accepting run $q_0 q_1 \dots q_n$ produces the output $w_{q_0} w_{q_1} \dots w_{q_n}$.
- **Homomorphisms and Rational Relations (Theorem 10.12):** A relation $R \subseteq \Sigma_1^* \times \dots \times \Sigma_m^*$ is rational if and only if there is a regular set P such that $(w_1, \dots, w_m) \in R \Leftrightarrow \exists v \in P [\pi_1(v) = w_1 \wedge \dots \wedge \pi_m(v) = w_m]$, where π_k is the projection (homomorphism) that preserves symbols from Σ_k and erases others.